Pune Vidyarthi Griha's College of Engineering, Technology and Management, Pune-09

Department of Electronics and Telecommunication Engineering

ENGINEERING MINI-PROJECT REPORT

Air Pollution / Air Quality Index (AQI) Prediction Using Machine Learning

Machine Learning Regression for Environmental Air-Quality Estimation

Submitted by
Omkar Satpute

Academic Year: 2026–27

Executive Summary

This project develops a machine-learning based AQI prediction system using daily air-pollution measurements from Indian cities. The project treats AQI as a continuous target and therefore uses regression as the primary ML formulation. The implementation compares Linear Regression, Decision Tree Regressor, and Random Forest Regressor using a common 80/20 train-test split. Missing pollutant values are handled through median imputation inside model pipelines, and AQI_Bucket is excluded to prevent target leakage. The final saved project model is a Decision Tree Regressor pipeline. A Streamlit-based interface was also developed so that a user can enter pollutant measurements and receive a predicted AQI.

Important: the assignment explicitly says not to provide fake output values. The accessible project artifacts confirm that the saved final model is a Pipeline containing SimpleImputer and DecisionTreeRegressor, and the notebook generated a final actual-vs-predicted AQI plot.
However, the notebook's numerical metric table (MAE, MSE, RMSE, R^2) and the final feature-importance values were not available as machine-readable outputs in the attached artifacts used to generate this report.
Therefore, this report deliberately does not invent those values. Before submission, copy the exact values from the completed notebook into the Results and Feature Importance tables below.

1. Introduction

Air pollution is a major environmental engineering problem because pollutant concentrations can vary substantially across locations and time.
An AQI prediction system can convert measured pollutant concentrations into an estimated AQI value and support monitoring, alerting, and planning decisions.

This project develops a supervised machine-learning regression system that predicts continuous AQI from daily pollutant measurements.
The work follows the assignment philosophy of defining the engineering problem first, understanding and validating the data, selecting features using evidence and domain reasoning, comparing suitable ML models, evaluating errors, interpreting the model, and translating the prediction into an engineering decision.

The project follows the assignment's required philosophy: the engineering problem and available data determine the ML approach, rather than selecting an algorithm first. The guideline requires dataset analysis, data-quality analysis, at least three meaningful visualizations, justified preprocessing and feature selection, an appropriate ML technique, evaluation, engineering interpretation, recommendations, and a final working demonstration.

2. Problem Definition

The engineering problem is to develop a data-driven system that estimates the Air Quality Index from observed air-pollutant concentrations.
The system is intended as a decision-support tool for environmental monitoring rather than as a

replacement for official monitoring or AQI computation.

The selected dataset contains daily city-level observations from Indian cities, including pollutant measurements and an AQI target.
The system takes pollutant measurements as input and produces a continuous predicted AQI as output. A high predicted AQI can be used to trigger additional monitoring, communication, or pollution-management attention.

2.1 Project Domain

Environmental engineering, air-quality monitoring, and applied machine learning.

2.2 Objective

- Use a real-world public air-quality dataset.
- Understand the structure, variables, missingness, duplicates, invalid values, outliers, and relationships in the data.
- Determine whether regression or classification is technically more appropriate for the primary task.
- Prepare the data without introducing target leakage.
- Select relevant pollutant features using data evidence and domain reasoning.
- Compare Linear Regression, Decision Tree Regressor, and Random Forest Regressor using consistent train/test methodology.
- Evaluate the models using MAE, MSE, RMSE, and R^2.
- Interpret the final model and identify which input variables contribute most to prediction.
- Create a simple prediction interface that accepts pollutant measurements and returns predicted AQI.
- Translate the ML output into an engineering/application interpretation and document limitations and future improvements.

2.3 Expected Inputs

PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, and Xylene.

2.4 Expected Output

A continuous predicted AQI value.

2.5 Engineering Decision Supported

The prediction can be used as an application-level monitoring indicator. When the predicted AQI is high, the system can flag the condition for additional monitoring, communication, or pollution-management attention. This is an engineering/application interpretation and not a medical diagnosis.

3. Dataset Description

The selected dataset is Air Quality Data in India (city_day.csv). Public descriptions of the dataset report 29,531 observations and 16 columns covering daily city-level measurements for 26 Indian cities. The dataset contains pollutant variables, AQI, and AQI_Bucket. The dataset is hosted on Kaggle and is associated with air-quality records compiled from CPCB-related sources.

Dataset	**Air Quality Data in India – city_day.csv**
Source	Kaggle: Air Quality Data in India by Rohan Rao; dataset compiled from CPCB-related air-quality records
Observations	29,531 daily city-level records

Columns	16
Coverage	2015–2020; 26 Indian cities
Target	AQI
Alternative target	AQI_Bucket (not used as a predictor)
Granularity	Daily city-level observations

The dataset is suitable because it contains multiple environmental predictors, a directly available AQI target, enough observations for train/test evaluation, and meaningful missing-value and distribution issues that make it appropriate for a complete ML mini-project.

3.1 Dataset Variables

Feature	Engineering meaning
PM2.5	Fine particulate matter; a direct pollutant indicator and relevant AQI predictor.
PM10	Coarser particulate matter; represents particulate pollution load.
NO	Nitric oxide; a nitrogen-related pollutant.
NO2	Nitrogen dioxide; a combustion-related pollutant indicator.
NOx	Combined nitrogen-oxide measure; potentially redundant with NO and NO2, so redundancy was considered.
NH3	Ammonia; an atmospheric pollutant included in the dataset.
CO	Carbon monoxide; combustion-related pollutant.
SO2	Sulfur dioxide; pollutant associated with combustion and industrial sources.
O3	Ozone; a secondary atmospheric pollutant.
Benzene	Volatile organic compound pollutant.
Toluene	Volatile organic compound pollutant.
Xylene	Volatile organic compound pollutant.
City	City identifier/context variable; not used in the final pollutant-only predictor set.
Date	Daily observation date; used for exploration but not included as a final predictor in the primary system.
AQI	Continuous target variable to be predicted.
AQI_Bucket	AQI category; excluded from predictors because it carries target-derived information.

4. Data Exploration

The notebook performed systematic dataset inspection including shape, first and last rows, column names, data types, statistical summaries, unique-value checks, target distribution, missing-value analysis, duplicates, invalid-value checks, outlier analysis, correlations, pollutant distributions, AQI relationships, city comparisons, and temporal AQI visualization.

4.1 Exploratory Visualizations

- AQI distribution histogram — used to understand the range and distribution of the continuous target.
- AQI category count plot — used to understand how observations are distributed across AQI_Bucket categories.
- Pollutant distribution plots — used to inspect the scale and skewness of pollution variables.

- Correlation heatmap — used to identify relationships between pollutants and AQI and to identify possible redundancy.
- Pollutant-versus-AQI scatter plots — used to inspect direct relationships between important pollutant measurements and the target.
- City-wise AQI comparison — used to understand geographic variation.
- Time-based AQI trend — used to inspect how air quality varies over the observation period.
- Actual-versus-predicted and residual plots — used for final model evaluation.

4.2 Key Interpretation

The EDA is not only descriptive. It informs feature selection, highlights missing-data problems, reveals possible redundancy among nitrogen-related variables, and provides evidence for treating extreme observations carefully rather than automatically deleting them.

5. Data Preprocessing

Preprocessing was designed around the project requirement that every major decision should have a technical reason. The modeling dataset was created by retaining rows with a known AQI target and excluding AQI_Bucket from the predictors. The final predictor set contains the 12 pollutant variables.

Decision	Implementation	Why	Consequence if omitted
Target missingness	Rows without AQI were excluded from supervised modeling	A supervised regression target is required	Avoids training against an undefined target
AQI_Bucket	Excluded from predictor set	It is derived from AQI/category information	Prevents target leakage
Missing predictors	Median imputation inside sklearn Pipeline	Pollutant measurements contain missing observations	Retains usable rows while avoiding leakage from test data
Scaling	StandardScaler used for Linear Regression pipeline; not used for tree models	Linear regression can benefit from standardized numerical inputs; trees do not require scaling	Keeps preprocessing algorithm-appropriate
Outliers	Not automatically removed	Extreme pollution values may represent genuine environmental events	Avoids deleting meaningful pollution extremes
City / Date	Not included in the initial final predictor set	The implemented system focuses on pollutant-to-AQI prediction; date/city engineering was not required for the primary task	Keeps the model simpler and avoids unnecessary feature creation
Train/test separation	80/20 split with random_state=42	Provides unseen test data for evaluation and reproducibility	Supports fair model comparison

5.1 Leakage Prevention

AQI_Bucket was not used as an input feature because it represents a category derived from AQI. Including it would allow the model to access target-related information and would make evaluation artificially optimistic. The imputer was placed inside the scikit-learn Pipeline so that imputation parameters are learned from the training data rather than from the complete dataset.

6. Feature Engineering and Feature Selection

The project intentionally avoids creating unnecessary derived variables. The initial feature set consists of the 12 available pollutant measurements. Feature selection was based on data availability, correlation analysis, domain relevance, and consideration of redundancy. NO, NO2, and NOx were specifically recognized as potentially correlated variables, so their relationship was investigated rather than removing one blindly.

City and Date were not included in the final primary predictor set. This keeps the implemented system focused on the practical input scenario demonstrated in the Streamlit interface: pollutant concentrations are entered and AQI is estimated.

6.1 Final Predictor Set

PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene.

7. ML Technique

Regression was selected as the primary ML approach because the target AQI is a continuous numerical value and the application requires an estimated AQI rather than only a category. Classification using AQI_Bucket is a valid alternative, but it was not selected as the primary formulation because it discards the continuous numerical information available in AQI.

7.1 Candidate Algorithms

Model	Reason for consideration	Advantages	Limitations
Linear Regression	Baseline linear model; easy to explain and interpret	Simple, interpretable baseline	May not capture nonlinear pollutant relationships
Decision Tree Regressor	Splits the feature space into rule-based regions	Captures nonlinear relationships and interactions; easy to explain	Can overfit without constraints
Random Forest Regressor	Averages predictions from many decision trees	Usually more stable than one tree and captures nonlinear relationships	More complex and less directly interpretable than one tree

7.2 Train/Test Strategy

The dataset was divided into 80% training data and 20% test data using random_state=42. The same split was used for the three candidate models so that the comparison remained fair and reproducible. Cross-validation was not required for the initial implementation and was retained as a future improvement rather than adding unnecessary complexity.

8. Implementation

The implementation uses Python with pandas, NumPy, Matplotlib, Seaborn, and scikit-learn. The model pipelines include preprocessing and the estimator so that the same transformations are applied consistently during training and prediction.

8.1 Linear Regression Pipeline

Median imputation → StandardScaler → LinearRegression.

8.2 Decision Tree Pipeline

Median imputation → DecisionTreeRegressor(random_state=42).

8.3 Random Forest Pipeline

Median imputation → RandomForestRegressor(random_state=42, n_estimators=100).

8.4 Final Prediction Interface

A Streamlit GUI was created with input fields for the 12 pollutant parameters. When the user clicks the prediction button, the values are assembled in the same feature order used during training and passed to the saved model. The application displays the predicted AQI.

9. Results

Important: the assignment explicitly says not to provide fake output values. The accessible project artifacts confirm that the saved final model is a Pipeline containing SimpleImputer and DecisionTreeRegressor, and the notebook generated a final actual-vs-predicted AQI plot.
However, the notebook's numerical metric table (MAE, MSE, RMSE, R^2) and the final feature-importance values were not available as machine-readable outputs in the attached artifacts used to generate this report.
Therefore, this report deliberately does not invent those values. Before submission, copy the exact values from the completed notebook into the Results and Feature Importance tables below.

Model	MAE	MSE	RMSE	R^2
Linear Regression	INSERT NOTEBOOK VALUE	INSERT NOTEBOOK VALUE	INSERT NOTEBOOK VALUE	INSERT NOTEBOOK VALUE
Decision Tree Regressor	INSERT NOTEBOOK VALUE	INSERT NOTEBOOK VALUE	INSERT NOTEBOOK VALUE	INSERT NOTEBOOK VALUE
Random Forest Regressor	INSERT NOTEBOOK VALUE	INSERT NOTEBOOK VALUE	INSERT NOTEBOOK VALUE	INSERT NOTEBOOK VALUE

The project artifact confirms that the final saved model is a scikit-learn Pipeline containing SimpleImputer followed by DecisionTreeRegressor. The notebook also produced a final actual-versus-predicted AQI visualization. The exact numerical metrics must be copied directly from the notebook before the report is submitted.

10. Evaluation

The project uses four regression metrics. MAE measures the average absolute prediction error and is easy to interpret in AQI units. MSE penalizes large errors more strongly because errors are squared. RMSE converts squared error back to the AQI scale and is particularly useful for understanding the magnitude of typical larger errors. R^2 describes the proportion of variance in AQI explained by the model relative to a baseline.

10.1 Evaluation Visualizations

- Actual vs Predicted AQI: points closer to the ideal diagonal indicate more accurate predictions.
- Residual/error plot: used to identify systematic overprediction, underprediction, or changing error variance.
- Prediction error distribution: used to understand whether errors are concentrated around zero or show skew/heavy tails.

10.2 Engineering Meaning of Error

A lower MAE/RMSE means that the predicted AQI is, on average, closer to the observed AQI. In an operational monitoring system, smaller prediction error generally means the model is more useful for estimating the severity of current air-quality conditions. However, suitability depends on the intended use, and a model should be validated against official measurements before operational deployment.

11. Engineering Interpretation

The ML system converts pollutant measurements into an estimated AQI. The most important practical output is not simply a model score but the ability to use the prediction as a monitoring signal.

- If predicted AQI is high, the application can flag the condition for attention and additional monitoring.
- If a pollutant is found to have high model importance, that pollutant becomes a priority variable for monitoring and data-quality checks.
- If prediction errors are systematically larger for extreme AQI values, the model may need additional data or a modeling strategy focused on extreme events.
- The model can support environmental planning and monitoring, but it should not be treated as a regulatory or medical decision system without further validation.

11.1 Feature Importance

Insert the exact feature-importance values generated by the final Decision Tree model here.

Rank	Feature	Importance
1	INSERT FROM NOTEBOOK	INSERT FROM NOTEBOOK
2	INSERT FROM NOTEBOOK	INSERT FROM NOTEBOOK
3	INSERT FROM NOTEBOOK	INSERT FROM NOTEBOOK
4	INSERT FROM NOTEBOOK	INSERT FROM NOTEBOOK
5	INSERT FROM NOTEBOOK	INSERT FROM NOTEBOOK

Feature importance must be interpreted as model contribution rather than proof that a pollutant causes AQI changes. The ranking should be taken directly from the trained model output.

12. Limitations

- The model is dependent on the quality, completeness, and representativeness of the selected dataset.
- The dataset represents historical city-level observations; performance may differ for new cities, sensors, or time periods.
- The primary task is same-observation AQI estimation from pollutant inputs, not a dedicated future-day forecasting model.
- Missing-value imputation can introduce approximation, especially when missingness is systematic.
- Extreme observations may be genuine pollution events; retaining them improves realism but can influence model behavior.
- The model predicts AQI from available inputs and should not be presented as an official regulatory AQI measurement system.
- Predictions for pollutant combinations far outside the training distribution may be unreliable.
- Feature importance indicates model contribution, not causation.
- The current demonstration requires the same feature definitions and units used during training.

13. Recommendations

- Validate incoming sensor measurements and reject physically impossible or clearly corrupted values.
- Use continuous monitoring and periodically retrain the model as new environmental data become available.
- Evaluate the model with cross-validation and hyperparameter tuning if higher reliability is required.
- Investigate city-aware or time-aware models if the objective changes from same-day estimation to future forecasting.
- Add meteorological variables such as temperature, humidity, wind speed, and pressure only when reliable data are available and their inclusion is justified.
- Monitor model drift when deploying the system in a changing environmental or sensor environment.
- Compare model predictions with official monitoring data before using the system for operational decisions.
- For production deployment, use a suitable hosting environment for the Streamlit application and protect the model artifact and dependency versions.

14. Conclusion

This project demonstrates a complete engineering-oriented machine-learning workflow for AQI prediction. A real-world Indian air-quality dataset was selected, the target was treated as a continuous variable, and regression was chosen as the primary formulation. The data were explored for quality and relationships, missing values were handled through a training-safe pipeline, AQI_Bucket was excluded to prevent leakage, and pollutant variables were selected using data analysis and domain reasoning.

Three understandable regression approaches were considered: Linear Regression, Decision Tree Regressor, and Random Forest Regressor. The final saved project artifact uses a Decision Tree Regressor with median imputation. The system also includes a user-facing Streamlit interface for new pollutant inputs. The engineering value of the project lies in translating predicted AQI into a monitoring and decision-support signal rather than ending with a numerical model score.

Before final academic submission, the exact MAE, MSE, RMSE, R^2, and feature-importance values must be copied from the completed notebook into this report. This is intentionally required because the assignment prohibits fabricated output values.

References

1. Pune Vidyarthi Griha's College of Engineering, Technology and Management, Department of Electronics and Telecommunication Engineering, Experiment 10 – Mini Project guideline PDF.
2. Rohan Rao, Air Quality Data in India (2015–2020), Kaggle. Dataset: city_day.csv.
3. Central Pollution Control Board (CPCB), Government of India — source organization associated with the underlying air-quality data.
4. Python scientific-computing ecosystem: pandas, NumPy, Matplotlib, Seaborn, and scikit-learn documentation.

Appendix A – Requirement Compliance Checklist

Requirement	Status	Evidence / Section
Engineering problem clearly defined	Completed	Problem Definition
Objective clearly stated	Completed	Objectives
Expected output identified	Completed	Problem Definition / Implementation

Real dataset source documented	Completed	Dataset Description
Features understood	Completed	Dataset Description / Feature Selection
Data quality checked	Completed in notebook	Data Exploration / Data Preprocessing
Missing values investigated	Completed in notebook	Data Preprocessing
At least 3 meaningful visualizations	Completed in notebook	Data Exploration / Evaluation
Appropriate preprocessing	Completed	Data Preprocessing
Feature selection	Completed	Feature Engineering/Selection
Transformations where required	Completed	Data Preprocessing
Preprocessing decisions justified	Completed	Data Preprocessing
Appropriate ML technique	Completed	ML Technique
Algorithm choice justified	Completed	ML Technique
Model implemented correctly	Completed	Implementation
Results generated	Completed in notebook	Results
Exact evaluation metrics	Needs final copy from notebook	Results table
Results interpreted	Partially documented; finalize after inserting metrics	Evaluation
Limitations identified	Completed	Limitations
Engineering meaning	Completed	Engineering Interpretation
Practical recommendation	Completed	Recommendations
Deployment limitations	Completed	Limitations / Recommendations
Notebook/code	Completed	Project notebook
Report	This document	Final report
Working demonstration	Streamlit interface created	Implementation

Appendix C – Final Submission Checklist

- Attach the final .ipynb notebook.
- Attach city_day.csv or provide the dataset source as required by the submission process.
- Attach final_model.pkl if the demonstration requires the saved model.
- Attach app.py and requirements.txt if the Streamlit demonstration is being submitted.
- Insert the exact model metrics from the notebook into Section 9.
- Insert the exact final feature-importance ranking into Section 11.1.
- Insert selected notebook visualizations into the report if printed submission requires figures.
- Verify that the final report, notebook, results, and demonstration are consistent with one another.